МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Выполнил
Е.А. Ерошин
студент группы ИИ-26

Проверил
К.В. Андренко
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 202

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:
- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.
2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:
- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.
4. Провести сравнительный анализ:
    - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
    - MSE + фиксированный шаг (данные из ЛР №1);
        - MSE + адаптивный шаг (данные из ЛР №2);
        - BCE + фиксированный шаг;
        - BCE + адаптивный шаг.
    - Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).
5. Визуализировать результаты:
    - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
    - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
    - Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.
6. Реализовать режим функционирования:
    - Пользователь вводит координаты x1,x2.
    - Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
    - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).
7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.
8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Допускается применение **математических** и **графических** библиотек

ML-библиотеки и **ML-фреймворки** использовать **нельзя** (например: scikit-learn, TensorFlow, PyTorch - запрещены)

Код программы:

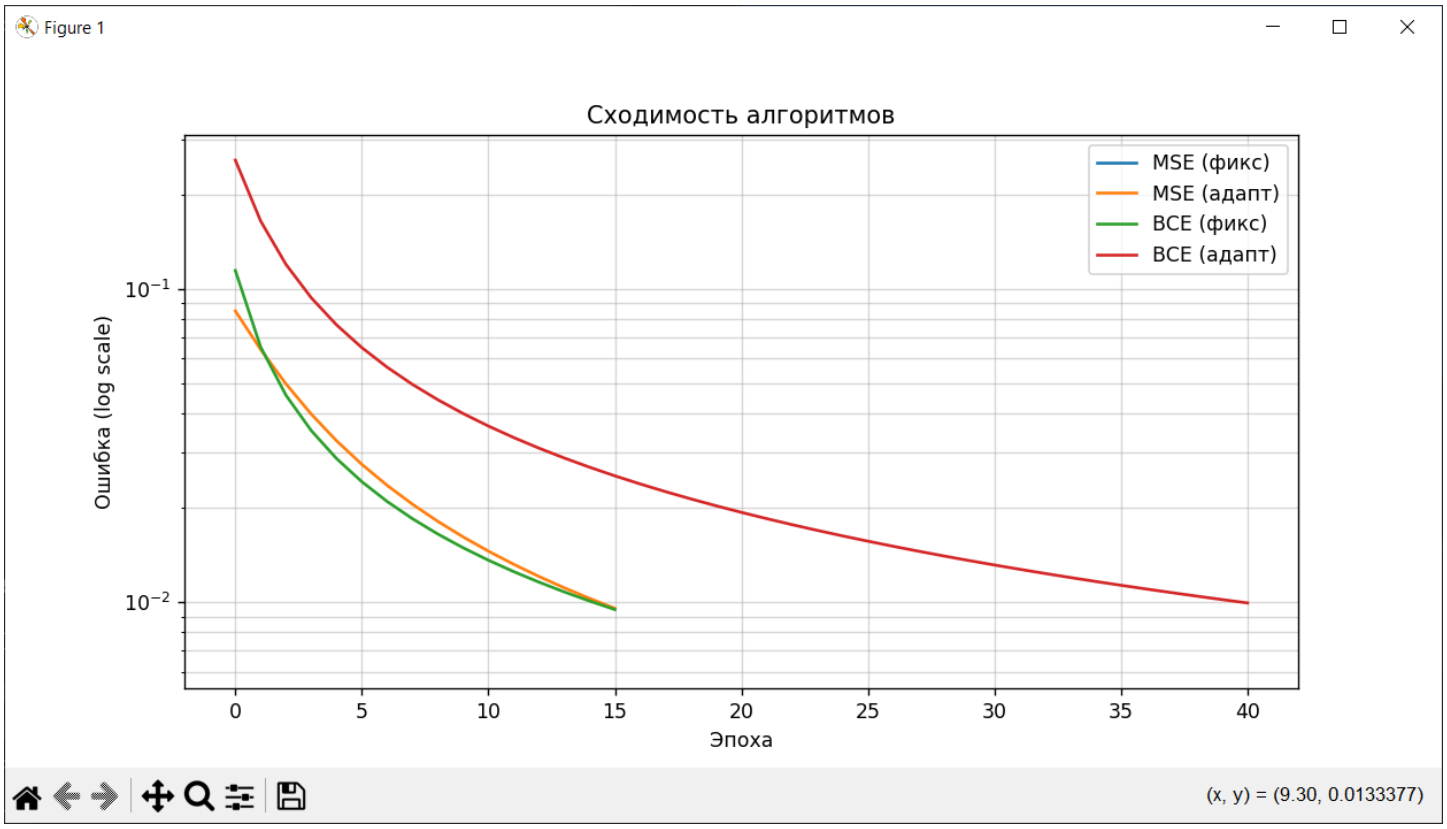
```
import numpy as np
import matplotlib.pyplot as plt


class BinaryClassifier:
    def __init__(self, features_count):
        # Инициализируем веса небольшими значениями для стабильности сигмоиды
        self.w = np.random.uniform(-0.1, 0.1, features_count + 1)

    def _add_bias(self, X):
        X = np.array(X, dtype=float)
        if X.ndim == 1:
            X = X.reshape(1, -1)
        bias = np.ones((X.shape[0], 1))
        return np.concatenate((bias, X), axis=1)

    def sigmoid(self, z):
        # Ограничиваем z, чтобы избежать переполнения exp
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def predict_proba(self, X):
        Xb = self._add_bias(X)
        return self.sigmoid(Xb @ self.w)

    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(int)

    def train_mse(self, X, y, alpha=0.1, adaptive=False, Ee=0.01, max_epochs=1000):
        Xb = self._add_bias(X)
        y = np.array(y, dtype=float)
        history = []

        for epoch in range(max_epochs):
```

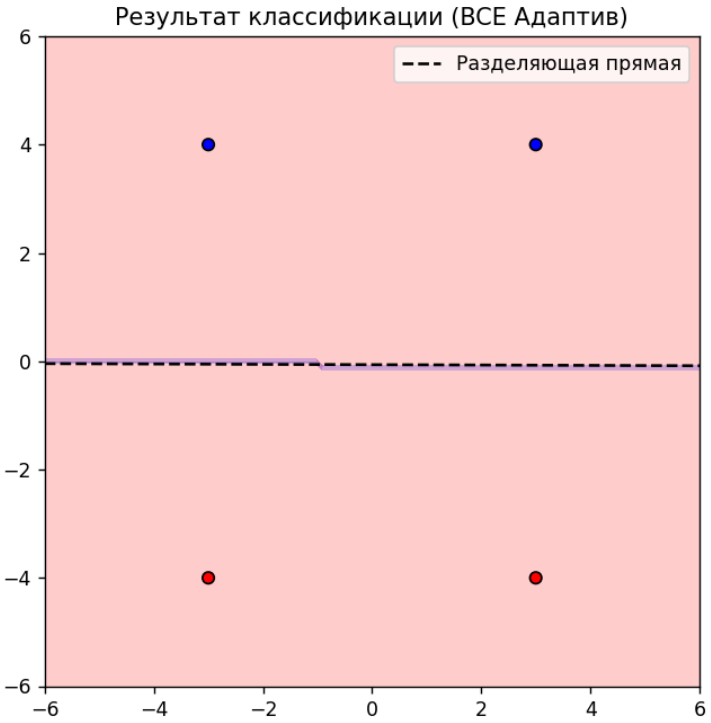
```
        for xi, target in zip(Xb, y):
            # Проход вперед
            z = np.dot(xi, self.w)
            y_hat = self.sigmoid(z)

            # Ошибка
            error = y_hat - target

            # Градиент MSE для сигмоиды: (y_hat - y) * sigmoid'(z) * x
            # sigmoid'(z) = y_hat * (1 - y_hat)
            gradient = error * (y_hat * (1 - y_hat)) * xi

            if adaptive:
                # Метод Качмажа с учетом производной (упрощенно)
                current_alpha = 1.0 / (np.dot(xi, xi) + 1e-6)
            else:
                current_alpha = alpha

            self.w -= current_alpha * gradient

        # Расчет средней ошибки по эпохе
        preds = self.sigmoid(Xb @ self.w)
        Es = 0.5 * np.mean((y - preds) ** 2)
        history.append(Es)

        if Es <= Ee:
            break
    return history

def train_bce(self, X, y, alpha=0.1, adaptive=False, Ee=0.01, max_epochs=1000):
    Xb = self._add_bias(X)
    y = np.array(y, dtype=float)
    history = []

    for epoch in range(max_epochs):
        for xi, target in zip(Xb, y):
            y_hat = self.sigmoid(np.dot(xi, self.w))

            # Градиент BCE для сигмоиды упрощается до: (y_hat - target) * xi
            gradient = (y_hat - target) * xi

            if adaptive:
                current_alpha = 1.0 / (np.dot(xi, xi) + 1e-6)
            else:
                current_alpha = alpha

            self.w -= current_alpha * gradient

        # BCE ошибка
        y_hat_all = self.sigmoid(Xb @ self.w)
        eps = 1e-12
        Es = -np.mean(
```

```
            y * np.log(y_hat_all + eps) +
            (1 - y) * np.log(1 - y_hat_all + eps)
        )
        history.append(Es)

        if Es <= Ee:
            break
    return history


# ====================== ДАННЫЕ И ОБУЧЕНИЕ ======================

X_data = np.array([[3, 4], [-3, 4], [3, -4], [-3, -4]])
y_data = np.array([1, 1, 0, 0])

# Запускаем обучение
model_mse_f = BinaryClassifier(2);
h_mse_f = model_mse_f.train_mse(X_data, y_data, alpha=0.5)
model_mse_a = BinaryClassifier(2);
h_mse_a = model_mse_a.train_mse(X_data, y_data, adaptive=True)
model_bce_f = BinaryClassifier(2);
h_bce_f = model_bce_f.train_bce(X_data, y_data, alpha=0.1)
model_bce_a = BinaryClassifier(2);
h_bce_a = model_bce_a.train_bce(X_data, y_data, adaptive=True)

# ====================== ВИЗУАЛИЗАЦИЯ ======================

plt.figure(figsize=(10, 5))
plt.plot(h_mse_f, label="MSE (фикс)")
plt.plot(h_mse_a, label="MSE (адапт)")
plt.plot(h_bce_f, label="BCE (фикс)")
plt.plot(h_bce_a, label="BCE (адапт)")
plt.yscale('log')  # Логарифмическая шкала лучше показывает сходимость
plt.xlabel("Эпоха")
plt.ylabel("Ошибка (log scale)")
plt.title("Сходимость алгоритмов")
plt.legend()
plt.grid(True, which="both", ls="-", alpha=0.5)
plt.show()

# Визуализация разделяющей линии (для лучшей модели)
xmin, xmax = -6, 6
ymin, ymax = -6, 6
grid_x, grid_y = np.meshgrid(np.linspace(xmin, xmax, 100), np.linspace(ymin, ymax, 100))
grid_points = np.c_[grid_x.ravel(), grid_y.ravel()]
grid_pred = model_bce_a.predict(grid_points).reshape(grid_x.shape)

plt.figure(figsize=(6, 6))
plt.contourf(grid_x, grid_y, grid_pred, alpha=0.2, colors=['red', 'blue'])
plt.scatter(X_data[:, 0], X_data[:, 1], c=y_data, cmap='bwr_r', edgecolors='k')

# Линия w0 + w1*x + w2*y = 0  =>  y = -(w0 + w1*x) / w2
```

```
w = model_bce_a.w
lx = np.array([xmin, xmax])
ly = -(w[0] + w[1] * lx) / w[2]
plt.plot(lx, ly, 'k--', label="Разделяющая прямая")

plt.xlim(xmin, xmax);
plt.ylim(ymin, ymax)
plt.title("Результат классификации (BCE Адаптив)")
plt.legend()
plt.show()

print(
    f"Эпох до сходимости:\nMSE фикс: {len(h_mse_f)}\nMSE адапт: {len(h_mse_a)}\nBCE фикс: {len(h_bce_f)}\nBCE адапт: {len(h_bce_a)}")
```

Результат работы программы





Вывод: Адаптивный шаг автоматически масштабирует величину изменения весов в зависимости от нормы входного вектора. Это повышает устойчивость алгоритма и уменьшает количество эпох обучения по сравнению с фиксированным шагом.

В ходе эксперимента установлено:

- адаптивный метод достигает критерия Es ≤ Ee быстрее
- кривая ошибки убывает более плавно
- обучение становится менее чувствительным к выбору начального α